Collision Harness

Submission write-up | Partial result

2/12 private checks	33/33 regression tests	$0 model usage

I built an original Node.js harness using only the standard library and Git. It assigns work to three independent worker identities across multiple OpenRouter models, uses separate Git worktrees, and checks candidates before serial integration. The public repository declares zero package dependencies and documents a one-command run. No Agent HQ or prebuilt orchestration framework is used.

The coordinator infers dependencies and file ownership, records task states and timestamps, checkpoints before requests and merges, and supports resuming interrupted work. The live plan was conservative and mostly serialized source changes. Operator review corrected merchant dependencies, changed providers and priorities, seeded an earlier logged candidate, and authored extra private behavior checks. All target-source changes in the result came from logged worker outputs; operator edits were to the harness, plan and checks. Those interventions are recorded in the event log.

The configured hard cap was $5, with a separate 45-dispatch limit. The completed run recorded 45 dispatches, including refusals and interrupted calls. Only free model IDs with zero catalog pricing were allowed; requests also enforced zero-price provider ceilings and disabled paid fallback. OpenRouter's final key usage is $0. Successful calls include token and cost evidence. Interrupted calls were reconciled against the key's zero recorded usage and their zero-price bounds, rather than invented generation-level usage.

The result is partial: 2 of 12 tasks pass author-written private behavior checks; the starter regression suite passes all 33 tests and the supplied test lock is unchanged. The hidden backlog uses different IDs from the supplied local acceptance files, so no organizer hidden-test pass is claimed. Failed candidates were rejected. Other tasks remain failed, blocked or unfinished. The detailed summary and logs are authoritative.

What went wrong: malformed JSON, placeholder code, output exhausted by reasoning, provider restrictions, upstream rate limits and timeouts consumed much of the request allowance. Initial regression checks alone missed incomplete behavior; the added private checks caught that. Next I would improve provider-aware routing, use smaller incremental edits, test inferred dependencies, and replace conservative whole-source ownership with better file analysis. The 10 offline harness tests pass, including collision protection, spend guards, merge rollback, honest status reporting and crash recovery.